JavaScript Lab — Practical 10

Course: JavaScript Programming Laboratory (Semester V)
Practical No.: 10
Aim: Load and display JSON data using fetch() and jQuery $.getJSON(); render results dynamically in a table.
Student Name: Samruddhi Kalbande
PRN: 24070521278
Institution: Symbiosis Institute of Technology (SIT), Nagpur

📌 Aim & Objectives

1. Master asynchronous data transfer using the modern JavaScript **Fetch API** (`async/await` , `fetch()` , `response.json()`).
2. Utilize **jQuery AJAX** helper methods, specifically `$.getJSON()` , for streamlined JSON ingestion.
3. Construct dynamic HTML table rows and cells from parsed JSON data structures using DOM manipulation.
4. Build a lightweight **Node.js HTTP REST backend** to handle `GET` and `POST` requests with file-based persistence.
5. Implement end-to-end case studies demonstrating JSON serialization (`JSON.stringify()`), deserialization (`JSON.parse()`), form processing, and dynamic dashboard metrics.

📂 Project Structure

```
pract 10/
├── README.md                    # Comprehensive Practical 10 documentation
├── README.pdf                   # PDF export of laboratory documentation
├── PRACTICAL 1/                 # REST API & Native Fetch Implementation
│   ├── index.html               # Fetch UI with dynamic table & student input form
│   ├── server.js                # Node.js HTTP server handling GET/POST /api/students
│   └── students.json            # Student records database
├── PRACTICAL 2/                 # jQuery $.getJSON() Implementation
│   ├── index.html               # Client-side jQuery consumer for local JSON data
│   └── students.json            # Sample dataset for jQuery parsing
├── CASESTUDY 1/                 # Employee Management System
│   ├── index.html               # Employee portal with form, JSON preview & table
│   ├── style.css                # Custom styling for dashboard cards & tables
```

```
│   └── employees.json          # Seed records for employee database
└── CASESTUDY 2/                # Online Food Order Dashboard
    ├── index.html              # Food order dashboard with jQuery $.getJSON & metrics
    └── orders.json             # Customer order transactions dataset
```

🔬 Theoretical Background

1. JavaScript Object Notation (JSON)

JSON is a lightweight, language-independent data-interchange format based on JavaScript object syntax. - **Data Types:** String, Number, Boolean, Array, Object, null. - **Serialization:** `JSON.stringify(object, replacer, space)` converts in-memory objects to a serialized string. - **Parsing:** `JSON.parse(text)` transforms JSON strings into native JavaScript objects.

2. Fetch API vs. jQuery $.getJSON()

Feature	Native Fetch API (`fetch()`)	jQuery (`$.getJSON()`)
Dependency	Built into modern ECMAScript / browsers	Requires jQuery library (`<script>` inclusion)
Syntax Style	Promise-based / `async/await`	Callback-based / Promise-compatible (`.done()` , `.fail()`)
Response Parsing	Explicit parsing required via `response.json()`	Automatically parses response JSON into JavaScript objects
Error Handling	Rejects on network failure; HTTP errors require `response.ok` check	Rejects directly on HTTP error codes (e.g., 404, 500) via `.fail()`
HTTP Methods	Configurable (`GET` , `POST` , `PUT` , `DELETE` , headers, body)	Dedicated for `GET` requests; `$.ajax()` used for other verbs

---

💻 Section 1: Practical 1 — REST API with Fetch & Node.js

Objective

Create a full-stack student management workflow where records are fetched asynchronously via `fetch()` and appended to a JSON file via `POST` requests to a Node.js HTTP server.

Backend Implementation (`server.js`)

- Listens on `http://localhost:3000`.
- Routes incoming requests:
- `GET /api/students`: Reads `students.json` and streams JSON payload with status 200.
- `POST /api/students`: Collects incoming data chunks, parses JSON body, assigns auto-incrementing ID, and saves to disk using `fs.writeFileSync`.
- `GET /`: Serves `index.html` static file using read streams.

```
// Handling POST in server.js
let body = '';
request.on('data', chunk => body += chunk);
request.on('end', () => {
    const students = JSON.parse(fs.readFileSync(file, 'utf8'));
    const student = JSON.parse(body);
    student.id = students.length ? students[students.length - 1].id + 1 : 1;
    students.push(student);
    fs.writeFileSync(file, JSON.stringify(students, null, 4));
    response.writeHead(201, { 'Content-Type': 'application/json' });
    response.end(JSON.stringify(student));
});
```

Client Frontend (`index.html`)

- **`loadStudents()`:** Executes `await fetch('/api/students')`, clears table body, and dynamically generates `<tr>` and `<td>` elements.
- **Form Submission Listener:** Captures name, department, and marks, submits payload as JSON via `fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body })`, and refreshes the table upon success.

💻 Section 2: Practical 2 — Asynchronous Ingestion with jQuery $.getJSON()

Objective

Ingest and render tabular data from a localized `students.json` file on button click using jQuery's simplified asynchronous method.

Key Logic

```
$("#loadBtn").click(function() {
    $.getJSON("students.json", function(data) {
        $("#tablebody").empty();
```

```
        $.each(data, function(index, student) {
            $("#tablebody").append(`
                <tr>
                    <td>${student.id}</td>
                    <td>${student.name}</td>
                    <td>${student.department}</td>
                    <td>${student.marks}</td>
                </tr>
            `);
        });
    }).fail(function() {
        alert("Error loading JSON data.");
    });
});
```

- Employs template literals for clean, declarative row injection into `#tablebody`.
- Implements `.fail()` callback to handle network or JSON parsing exceptions gracefully.

🏢 Section 3: Case Studies

Case Study 1: Employee Management System

- **Dual Data Handling:** Blends initial external JSON data (`employees.json`) with interactive user submissions collected via form controls.
- **Real-Time JSON Inspector:** Displays formatted JSON strings in a `<pre id="jsonData">` element using `JSON.stringify(employee, null, 2)` to give visual feedback on active data objects.
- **Merged Array Rendering:** Utilizes ES6 spread operator (`displayEmployees([...employees, ...newEmployees])`) to ensure seamless data unification.

Case Study 2: Online Food Order Dashboard

- **jQuery Dashboard Architecture:** Pulls customer transaction data (`orders.json`) including Order ID, Customer Name, Food Item, Quantity, and Price.
- **Dynamic Table Generation:** Traverses order lists with `$.each()` to construct styled order records.
- **Financial Metric Aggregations:** Computes total items ordered, total revenue, and average spend across all processed orders.
- **Error Shielding:** Incorporates error messaging elements that notify users if network or server errors interrupt the data pipeline.

🎯 Learning Outcomes

- Proficient in handling asynchronous data exchange using both standard Fetch API and jQuery shorthand methods.
- Understood full-stack REST interactions with Node.js and dynamic DOM rendering.
- Gained hands-on experience structuring real-world JSON models, error handling pipelines, and tabular presentation.